

O2O 商铺食品安全相关评论发现

刘 x 2025.....1
刘 xx 2025.....2
Illusionna 2025.....4

15:02, Saturday 13th December, 2025

I 队伍名称



图 1: 电子矿工

II 比赛排名

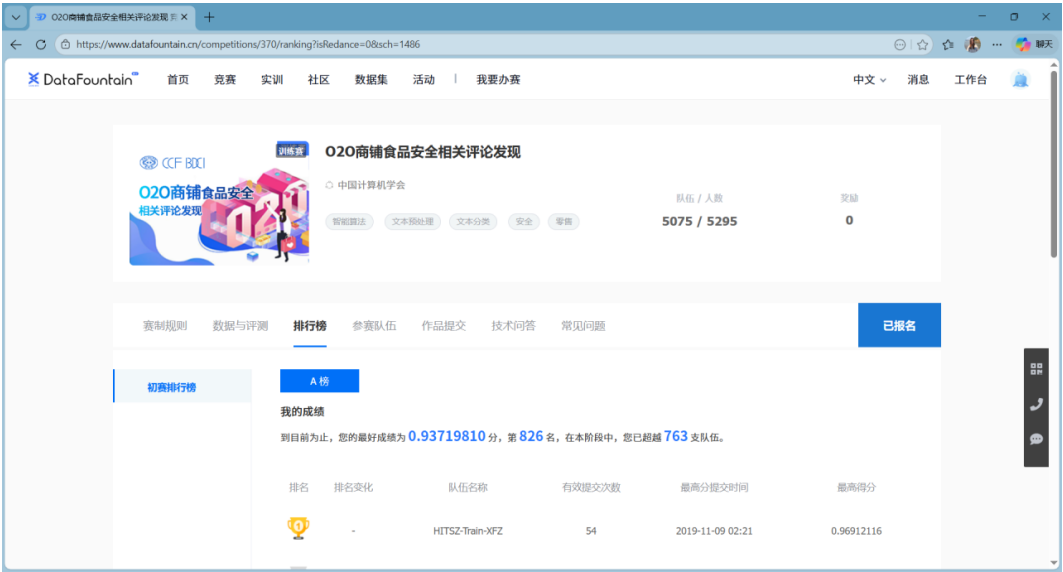


图 2: 成绩 0.9372 排名 826

III 算法介绍

(a) Gradient Boosting

submission_gradientboosting.csv ↓		
所在赛程	状态 / 得分	提交时间
初赛 - A榜	0.88383836000 查看日志	2025/12/12 20:18
备注: 无备注信息		

图 3: Gradient Boosting 模型 0.8838 得分

Gradient Boosting 是一种用于回归和分类问题的集成学习技术。它与其他集成方法（譬如随机森林）构建一堆相互独立的树截然不同，它采用的是 Boosting 提升策略。其核心思想是，通过迭代训练一系列弱学习器（譬如决策树），每一个新的学习器都致力于纠正前一个学习器的残差。最终将这些弱学习器加权组合，形成一个强预测模型。即 Gradient Boosting 的每一轮迭代模型都是在学习拟合上一轮模型的残差。

GBM 的最终模型是一个加法模型，由 M 个基函数弱学习器组成。假设我们的模型为 $F(x)$:

$$F_M(x) = \sum_{m=1}^M \gamma_m h_m(x)$$

其中， $h_m(x)$ 是第 m 个弱学习器，通常是 CART 回归树， γ_m 是第 m 个学习器的权重步长。

我们需要最小化损失函数 $L[y, F(x)]$ ，在第 m 轮迭代时，我们希望找到一个 $h_m(x)$ 和 γ_m ，以使得：

$$\arg \min_{\gamma, h(x)} \{L[y, F_{m-1}(x) + \gamma_m h_m(x)]\}$$

我们利用梯度下降法在函数空间中进行优化，为了最小化损失函数，我们计算损失函数关于当前模型预测值的负梯度，这个负梯度被称为伪残差，记为 r_{im} ，对于第 i 个样本在第 m 轮迭代：

$$r_{im} = - \left\{ \frac{\partial L[y_i, F(x_i)]}{\partial F(x_i)} \right\}_{F(x)=F_{m-1}(x)}$$

例如损失函数是平方误差 MSE，则负梯度就是标准的残差：

$$L = \frac{1}{2} [y - F(x)]^2 \implies -\frac{\partial L}{\partial F(x)} = y - F(x)$$

负梯度指向损失下降最快的方向，能尽可能最快地收敛，算法整体流程如下：

Algorithm 1 随机梯度增强求解分类问题

- 1: **Input:** 训练数据 $\{(x_i, y_i) | i = 1, 2, 3, \dots, N\}$ 和损失函数 $L[y, F(x)]$
 - 2: **Parameters:** $M = 200, \nu = 0.1, \text{subsample} = 0.8$
 - 3: **Initialize:** $F_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$
 - 4: **for** $m = 1 \rightarrow M$ **do**
 - 5: **Subsampling:** 随机采样 80% 形成子集 $\mathcal{S}_m \subseteq \{1, \dots, N\}$
 - 6: **Pseudo-Residuals:** 对于子集的每一个元素 $i \in \mathcal{S}_m$, 计算 $r_{im} = -\frac{\partial L[y_i, F_{m-1}(x_i)]}{\partial F_{m-1}(x_i)}$
 - 7: **Fit Tree:** 使用 $\mathcal{S}_m$ 拟合一个新的基学习器 (譬如回归树) $h_m(x)$
 - 8: (约束 $\text{depth} \leq 10, \text{leaf_samples} \geq 10, \text{split_features} \approx \sqrt{d}$)
 - 9: **Step Size:** 求解最佳步长 $\gamma_m = \arg \min_{\gamma} \sum_{i=1}^{|\mathcal{S}|} L[y_i, F_{m-1}(x_i) + \gamma h_m(x_i)]$
 - 10: **Update:** 最新模型 $F_m(x) = F_{m-1}(x) + \nu \cdot \gamma_m h_m(x)$
 - 11: (注意 ν 代表学习率 Learning Rate, 用于控制过拟合)
 - 12: **end for**
 - 13: **Output:** 最终分类器模型 $F_M(x)$
-

Gradient Boosting 通常能提供优于随机森林等算法的预测精度, 支持不限于最小二乘的各种可微损失函数, 对数值型和类别型特征都有较好的处理能力, 但由于模型是顺序构建的, 后者依赖前者, 训练速度较慢, 难以完全并行化, 并且如果不加正则化或者树的数量过多, 很容易过拟合。

(b) RNN

submission_rnn.csv ↓		
所在赛程	状态 / 得分	提交时间
初赛 - A榜	0.89244854000 查看日志	2025/12/12 20:04
备注: 无备注信息		

图 4: RNN 模型 0.8924 得分

Recurrent Neural Network 循环神经网络是一类用于处理序列数据的神经网络。与传统神经网络不同, RNN 引入了状态变量 (隐藏状态) 来记录过去的信息, 从而能够捕捉数据中的时间依赖性。

RNN 的核心在于其循环单元。假设输入序列为 $X = \{x_1, x_2, \dots, x_T\}$, 在时刻 t , RNN 的计算逻辑如下。

隐藏状态的更新由当前输入和上一时刻的隐藏状态共同决定:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

当前时刻的输出由当前的隐藏状态决定：

$$y_t = W_{hy}h_t + b_y$$

- $x_t \in \mathbb{R}^d$: t 时刻的输入向量；
- $h_t \in \mathbb{R}^h$: t 时刻的隐藏状态记忆；
- $y_t \in \mathbb{R}^q$: t 时刻的输出向量；
- W_{xh}, W_{hh}, W_{hy} : 权重矩阵；
- b_h, b_y : 偏置向量；
- $\sigma(\cdot)$: 激活函数（通常为 tanh 或 ReLU）。

Algorithm 2 RNN 前向传播算法

Require: 输入序列 $X = \{x_1, x_2, \dots, x_T\}$, 初始隐藏状态 h_0 , 网络参数 $W_{xh}, W_{hh}, W_{hy}, b_h, b_y$

Ensure: 输出序列 $Y = \{y_1, y_2, \dots, y_T\}$, 最终隐藏状态 h_T

```

1: 初始化输出列表  $Y \leftarrow []$ 
2:  $h_{prev} \leftarrow h_0$                                 ▷ 初始化上一时刻状态
3: for  $t = 1 \rightarrow T$  do
4:   Step 1: 计算隐藏状态
5:    $linear\_h \leftarrow W_{hh} \cdot h_{prev} + W_{xh} \cdot x_t + b_h$ 
6:    $h_t \leftarrow \tanh(linear\_h)$                         ▷ 使用 Tanh 激活函数
7:   Step 2: 计算输出
8:    $o_t \leftarrow W_{hy} \cdot h_t + b_y$ 
9:    $y_t \leftarrow \text{Softmax}(o_t)$                         ▷ 假设用于分类任务
10:  Step 3: 保存结果与更新状态
11:  将  $y_t$  添加到  $Y$ 
12:   $h_{prev} \leftarrow h_t$ 
13: end for
14: return  $Y, h_{prev}$ 

```

(c) SVM、XGBoost、GBRT

submission_sxg.csv ↓		
所在赛程	状态 / 得分	提交时间
初赛 - A榜	0.89952150000 查看日志	2025/12/12 22:19
备注: 无备注信息		

图 5: SVM、XGBoost、GBRT 混合模型 0.8995 得分

SVM 支持向量机是一种监督学习算法，主要用于分类和回归分析。其核心思想是在特征空间中寻找一个最优的超平面，使得不同类别的样本被该超平面分开，且两类样本到超平面的间隔最大化。

对于线性可分的数据集 $D = \{(x_i, y_i) | i = 1, 2, 3, \dots, N\}$ ，其中 $x_i \in \mathbb{R}^n$ ， $y_i \in \{+1, -1\}$ 。目标是找到一个超平面 $w^\top x + b = 0$ ，为了使分类间隔最大化，需求解优化问题：

$$\begin{cases} \min_{w,b} & \frac{1}{2} \cdot \|w\|^2 \\ \text{s.t.} & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \dots, N \end{cases}$$

在现实数据中，通常很难做到线性可分，为此引入松弛变量 $\xi_i \geq 0$ 和惩罚参数 C ，允许少量样本分类错误：

$$\min_{w,b,\xi} \left\{ \frac{1}{2} \cdot \|w\|^2 + C \sum_{i=1}^N \xi_i \right\}$$

对于非线性问题，SVM 通过核函数 $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$ 将低维空间的非线性问题映射到高维空间使其线性可分，常用的核函数包括：

- 线性核；
- 多项式核；
- 高斯核 $K(x_i, x_j) = e^{(-\gamma \|x_i - x_j\|^2)}$ 。

XGBoost 是 Gradient Boosted Regression Trees 的一种高效工程实现和算法改进，它在 GBRT 的基础上进行了多项优化，不仅在算法层面引入了二阶泰勒展开和正则化项，还在系统层面实现了并行计算和缓存优化。

不同于传统 GBRT 仅利用一阶导数，XGBoost 对损失函数进行了二阶泰勒展开：

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^N \left\{ L[y_i, \hat{y}_i^{(t-1)}] + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right\} + \Omega(f_t)$$

其中， g_i 是一阶梯度， h_i 是二阶梯度， $\Omega(f_t)$ 是正则化项，用于控制模型复杂度，防止过拟合：

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

其中， T 为叶子节点数量， w 为叶子节点权重向量。XGBoost 内置 L1 和 L2 正则化项，提升模型泛化能力，借鉴随机森林，支持特征子采样，减少计算量并防止过拟合，能够自动学习出缺失值的默认分裂方向。

我们的模型采用堆叠泛化策略，将三个基学习器 SVM、XGBoost、GBRT 进行级联。整个架构分为两层：

- 第一层由 SVM、GBRT 和 XGBoost 组成，它们并行地对原始特征进行学习，并分别输出预测结果；
- 第二层为集成学习的多数投票原则，它将第一层三个模型的输出作为新的特征输入，学习投票决定这些预测值以获得最终结果。

基模型预测训练三个基模，第一层的输出向量为：

$$Z = [z_1, z_2, z_3]^\top = [f_{\text{SVM}}(x), f_{\text{GBRT}}(x), f_{\text{XGB}}(x)]^\top$$

再通过 Majority Voting 多数硬投票分类器：

$$\hat{y} = \arg \max_{j \in \{1, 2, \dots, K\}} \sum_{t=1}^T \mathbb{I}[h_t(x) = c_j]$$

其中， $\mathbb{I}[h_t(x) = c_j]$ 统计有多少个分类器预测结果是类别 c_j 。

(d) BERT

submission_bert.csv ↓		
所在赛程	状态 / 得分	提交时间
初赛 - A榜	0.93719810000 查看日志	2025/12/13 14:35
备注：无备注信息		

图 6: BERT 模型 0.9372 得分

BERT 核心优势在于利用 Transformer 的 Encoder 结构进行深度的双向语言表示学习。与传统的单向语言模型 RNN 不同，BERT 能够同时利用上下文信息，从而更精准地捕捉评论文本中的语义特征。

核心架构与输入表示 BERT 基于多层双向 Transformer 编码器。在本任务中，我们采用了预训练的 BERT 模型作为基座。模型的输入不仅仅是简单的词向量，而是由三部分嵌入求和得到，确保了模型对语义、分段和位置信息的全面感知：

$$E = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}}$$

- Token Embeddings 词嵌入：单词本身的向量表示；
- Segment Embeddings 段嵌入：用于区分句子对，在本单句分类任务中主要辅助模型理解结构；
- Position Embeddings 位置嵌入：由于 Transformer 不具备 RNN 的循环结构，需引入位置编码来捕捉序列顺序信息。

BERT 核心机制是多头自注意力机制，BERT 摒弃了循环网络结构，完全依赖注意力机制来捕捉长距离依赖关系。通过计算 Query (Q)、Key (K) 和 Value (V) 之间的相互作用，模型能够关注到句子中对分类结果贡献最大的关键词：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

下游任务微调为了适配本次“O2O 商铺食品安全相关评论发现”的二分类任务，我们在 BERT 输出的 [CLS] 标记，代表整个句子的语义表示，然后接入了一个全连接层 Feed Forward Network 和 Softmax 分类器。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

掩码预测的概率计算：

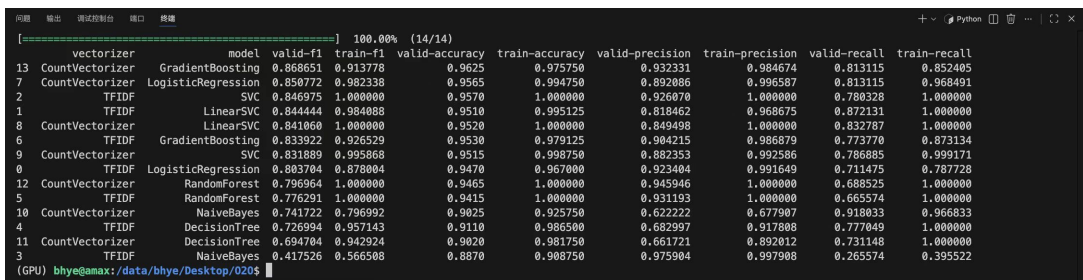
$$P(w_i|w_{\setminus i}) = \frac{e^{(E_{w_i} \cdot x_i)}}{\sum_j e^{(E_{w_j} \cdot x_i)}}$$

相比于 SVM 和 Gradient Boosting 等传统机器学习算法，BERT 通过海量文本的预训练已经掌握了丰富的语言学知识，因此在面对复杂的评论语境时泛化能力更强。

IV 总结

本次比赛中，我们团队针对“O2O 商铺食品安全相关评论发现”任务，从传统的统计机器学习到前沿的深度学习模型进行了系统的实验与迭代。通过对比不同特征提取方法和模型架构，我们得出以下结论：

- 特征工程阶段：CountVectorizer 优于 TF-IDF 在早期的特征工程尝试中，我们对比了 CountVectorizer 和 TF-IDF 两种文本特征提取方式。实验数据表明，在结合 Gradient Boosting、SVM 等分类器时，CountVectorizer 的表现普遍优于 TF-IDF。这可能是因为短文本评论中，词频特征比逆文档频率更能捕捉到关键的负面评价词汇。
- 传统机器学习与集成策略在单一机器学习模型中，Gradient Boosting 表现最佳，达到了 0.8838 的分数，显著优于逻辑回归和朴素贝叶斯等基准模型。为了进一步提升性能，我们采用了 Stacking 堆叠泛化策略，将 SVM、XGBoost 和 GBRT 进行级联。该混合模型利用了不同算法的差异性互补，将分数提升至 0.8995。
- 深度学习模型的突破鉴于传统模型在语义理解上的局限性，我们引入了深度学习模型：
 - RNN 利用 LSTM/GRU 单元捕捉序列的时间依赖性，得分为 0.8924，略优于传统集成模型。
 - BERT 凭借其强大的双向上下文建模能力和注意力机制，BERT 模型在所有方案中脱颖而出，取得了 0.9372 的最高分。



	vectorizer	model	valid-f1	train-f1	valid-accuracy	train-accuracy	valid-precision	train-precision	valid-recall	train-recall
13	CountVectorizer	GradientBoosting	0.868651	0.913778	0.9625	0.975750	0.932331	0.984674	0.813115	0.852405
7	CountVectorizer	LogisticRegression	0.850772	0.982338	0.9565	0.994750	0.892086	0.996587	0.813115	0.968491
2	TFIDF	SVC	0.846975	1.000000	0.9570	1.000000	0.926070	1.000000	0.788328	1.000000
1	TFIDF	LinearSVC	0.844444	0.984088	0.9510	0.995125	0.818462	0.968675	0.872131	1.000000
8	CountVectorizer	LinearSVC	0.841060	1.000000	0.9520	1.000000	0.849498	1.000000	0.832787	1.000000
6	TFIDF	GradientBoosting	0.833922	0.926529	0.9530	0.979125	0.984215	0.986879	0.773770	0.873134
9	CountVectorizer	SVC	0.831889	0.995868	0.9515	0.998750	0.882353	0.992586	0.786885	0.999171
10	TFIDF	LogisticRegression	0.803704	0.878004	0.9470	0.967000	0.923404	0.991649	0.711475	0.787728
12	CountVectorizer	RandomForest	0.796564	1.000000	0.9465	1.000000	0.945946	1.000000	0.685525	1.000000
5	TFIDF	RandomForest	0.776291	1.000000	0.9415	1.000000	0.931193	1.000000	0.655574	1.000000
10	CountVectorizer	NaiveBayes	0.741722	0.706992	0.9025	0.925750	0.822222	0.677907	0.918833	0.966833
4	TFIDF	DecisionTree	0.726994	0.957143	0.9110	0.986500	0.682997	0.917808	0.777049	1.000000
11	CountVectorizer	DecisionTree	0.694704	0.942924	0.9020	0.981750	0.661721	0.892012	0.731148	1.000000
3	TFIDF	NaiveBayes	0.417526	0.566508	0.8870	0.908750	0.975904	0.997908	0.265574	0.395522

图 7: 按照验证集 F1 指数排序后的机器学习算法

```
Gradient Boosting -----> 0.8838
RNN -----> 0.8924
SVM + XGBoost + GBRT ----> 0.8995
BERT -----> 0.9372
```

我们的模型迭代路径呈现出清晰的性能提升趋势，最终我们选择 BERT 作为最高分提交结果，并在最终排名中位列第 826 名，未来的改进方向可考虑结合 BERT 的特征嵌入与 GBRT 的分类能力，或者尝试更轻量的 DistilBERT 以提升推理速度。